

Lecture 10: Advanced DOM Manipulation and Events

Connecting JavaScript to HTML:

- To enable DOM manipulation, the JavaScript file must be explicitly linked to the HTML document.
- This is done using the script tag: `<script src="./first.js"></script>`.

Event Handling:

DOM elements can be programmed to change dynamically based on user interactions, known as events (e.g., mouse clicks, double clicks, keyboard inputs).

1. Legacy Event Methods:

Historically, events were attached directly within HTML or by assigning a function to an object property.

- **Inline HTML:** Adding attributes like `onclick="handleClick()"` directly to the HTML tag.
- **Property Assignment:** Assigning a function directly to the element's property in JavaScript. The property is initially set to null and executes the function only when the click occurs.

```
const element = document.getElementById('first');
```

```
element.onclick = function handleClick() {  
  element.textContent = "Hello Coder Army";  
  element.style.backgroundColor = "skyblue";  
}
```

2. Modern Event Listeners:

The modern standard utilizes the `.addEventListener()` method, which prevents inconsistencies found in older methods.

```
const element = document.getElementById('first');
```

```
element.addEventListener('click', () => {  
  element.textContent = "Hello Coder Army";  
})
```

Behind the Scenes: The global window object manages these events and is responsible for executing the attached functionality when the specific interaction is detected.

Creating and Inserting New Elements:

JavaScript can generate entirely new HTML elements from scratch and inject them into the live document.

Element Creation:

`document.createElement("tagName")`: Generates a brand new element object.

Insertion Methods:

Once created and populated with text or styles, the element must be placed into the DOM structure using specific methods.

Method	Behavior
<code>.after(newElement)</code>	Inserts the new element immediately after the targeted sibling element.
<code>.append(newElement)</code>	Inserts the new element immediately before the targeted sibling element.
<code>.prepend(newElement)</code>	Injects the new element at the very end of a parent container.
<code>.before(newElement)</code>	Injects the new element at the very beginning of a parent container.

Note: To insert an element in the middle of a list, specific children can be targeted (e.g., `ul.children[1].after(listItem3);`).

Modifying Attributes and Classes:

1. The `className` vs. `classList` Property:

- Because `class` is a strictly reserved keyword in JavaScript, the property `.className` is used to manipulate CSS classes directly.
- However, assigning multiple classes via `.className` requires messy string concatenation (`+=`), which creates confusion.
- **The Fix:** Using `.classList.add("className")` is the preferred, cleaner method to attach additional classes safely.

2. Standard Attributes:

General attributes are managed using dedicated getter and setter methods.

```
console.log(newElement.getAttribute("class"));

newElement.setAttribute("name", "rohit");
```

Note: When using `.getAttribute()`, the exact string `"class"` can be safely used without triggering keyword conflicts.

DOM Performance and the Document Fragment:

When receiving large datasets from a backend (like an array of food items), injecting them into the DOM efficiently is critical for performance.

- **The Problem:** Iterating over an array and appending elements to the live DOM one by one forces the browser to constantly recalculate and update the UI. Behind the scenes, this heavy calculation can cause the website to hang or crash entirely.
- **The Solution (DocumentFragment):** A document fragment acts as an invisible, lightweight container. Elements are appended to this fragment during the loop, and the entire fragment is appended to the main DOM exactly once at the end.

Method 1: Optimized Code using DocumentFragment

```
const foods = ["Milk", "Soya", "Jalebi", "Samosa", "Rasgulla"];
const ul = document.getElementById("ul");

const fragment = document.createDocumentFragment();

for (const food of foods) {
  const li = document.createElement("li");
  li.textContent = food;
  fragment.appendChild(li);
}
ul.appendChild(fragment);
```

Method 2: Alternative Performance Strategy (Using Arrays)

Elements can also be collected into a standard JavaScript array during a loop and appended all at once.

```
const arr = [];
```

```
for(const food of foods) {  
  const li = document.createElement('li');  
  li.textContent = food;  
  arr.push(li);  
}  
ul.append(...arr);
```